In-Depth Analysis of Homomorphic Encryption Libraries

Final Report

Authors: Itai Zloclzower, Jacob Shemesh, Tomer Ovadia, Yoni Glickstein

Course: Confidential Computing, Ben-Gurion University of the Negev

Date: June 2026

1. Introduction

Homomorphic Encryption (HE) is a cryptographic technique that allows computations to be performed directly on encrypted data. In a regular encryption scheme, data must be decrypted before it can be processed, creating a security gap. HE addresses this by enabling a server or cloud service to compute over ciphertexts without learning the underlying plaintext values.

This project compares two HE library implementations — **TenSEAL** and **OpenFHE** — through a set of basic encrypted computations. Both libraries implement the **CKKS scheme** (Cheon–Kim–Kim–Song), which supports approximate arithmetic on real numbers and is well-suited for data analytics workloads.

The focus is on understanding the practical cost of using HE libraries in terms of **runtime**, **memory usage**, **approximation error**, and **implementation complexity**, compared to equivalent plaintext computation.

2. Research Goal

Research Question: What is the practical overhead of performing basic computations using two different homomorphic encryption implementations, compared to equivalent plaintext computation?

The comparison focuses on operations common in data analysis: addition, multiplication, summation, average calculation, and dot product.

3. Libraries and Scheme

3.1 CKKS Scheme

Both libraries use the CKKS (Cheon–Kim–Kim–Song) homomorphic encryption scheme, which supports approximate arithmetic over real and complex numbers. Unlike the BFV/BGV schemes that operate on exact integers, CKKS is designed for applications where small approximation errors are acceptable — such as machine learning inference and statistical analysis.

Key CKKS concepts: - **Polynomial modulus degree** determines security level and capacity for computation. - **Scaling factor** controls the precision of encoded real numbers. - **Multiplicative depth** limits the number of sequential multiplications before noise overwhelms the result.

3.2 TenSEAL (v0.3.16)

TenSEAL is a Python library built on top of Microsoft SEAL. It provides a high-level, Pythonic API for encrypted tensor operations. Configuration used: - `poly_modulus_degree = 8192` - `coeff_mod_bit_sizes = [60, 40, 40, 60]` - `global_scale = 2^40`

TenSEAL's main advantage is its simplicity — encrypted vectors support Python operators (`+`, `*`, `.sum()`), making the code nearly identical to plaintext equivalents.

3.3 OpenFHE (v1.5.1)

OpenFHE is an open-source C++ library with Python bindings, maintained by a consortium of academic institutions. It provides a lower-level API with more control over cryptographic parameters. Configuration used: - `multiplicative_depth = 3` - `scaling_mod_size = 50` - `batch_size` = next power of two of the input vector length

OpenFHE requires more explicit API calls (e.g., `EvalAdd`, `EvalMult`, `EvalSum`) and manual management of key generation steps (multiplication keys, summation keys).

4. Implementation

4.1 Mandatory Operations

All mandatory operations from the proposal were implemented for both libraries:

Operation	Description
Key Generation	Crypto context creation, key pair generation, evaluation keys
Encryption	Encoding a plaintext vector into a ciphertext
Addition	Encrypted vector + encrypted vector (element-wise)
Multiplication	Encrypted vector × encrypted vector (element-wise)
Summation	Sum of all elements in an encrypted vector
Average	Sum divided by vector length (plaintext scalar division)
Decryption	Recovering the plaintext result from a ciphertext

4.2 Optional Extensions (All Completed)

Extension	Description
Dot Product	Element-wise multiplication followed by summation
Weighted Score	Healthcare use case: weighted average over encrypted patient data
Scaling Test	Benchmarks at vector sizes 5, 10, 50, 100, 500

4.3 Practical Limitation: Batch Size

During implementation, we discovered that **OpenFHE requires the batch size to be a power of two**, while TenSEAL accepts arbitrary vector lengths. This required adding a helper function to round up the batch size, which is a practical usability difference between the two libraries.

5. Experimental Results

All benchmarks were run on the same machine under identical conditions. The input vector for the main benchmark is `[1.0, 2.0, 3.0, 4.0, 5.0]`.

5.1 Runtime Comparison

Operation	Plaintext	TenSEAL (ms)	OpenFHE (ms)
Encrypt	—	8.165	39.467
Add	—	0.398	0.217
Multiply	—	3.556	98.337

Operation	Plaintext	TenSEAL (ms)	OpenFHE (ms)
Sum	—	13.506	295.441
Average	—	13.907	304.985
Dot Product	—	6.201	368.075
Decrypt	—	2.597	128.948
Total (all HE ops)	**0.011**	**48.330**	**1,235.470**
Overhead vs plaintext	**1×**	**4,433×**	**113,325×**

Key findings: - TenSEAL is approximately **25× faster** than OpenFHE across all operations. - Both libraries show that **summation and average** are the most expensive operations, as they require rotations across all ciphertext slots. - **Addition** is the cheapest encrypted operation in both libraries (sub-millisecond). - The overhead vs. plaintext is enormous: even TenSEAL is ~4,400× slower, and OpenFHE is ~113,000× slower.

5.2 Memory Usage

Operation	Plaintext (KB)	TenSEAL (KB)	OpenFHE (KB)
Encrypt	0.34	1.30	0.40
Add	—	0.38	0.06
Multiply	—	0.37	0.06
Sum	—	0.38	0.06
Average	—	0.79	0.67
Dot Product	—	0.68	0.45

Key findings: - Python-traced memory (via `tracemalloc`) captures only the Python-side allocations; the underlying C/C++ libraries allocate most memory outside Python's tracking. - OpenFHE shows **lower Python-side memory** for most operations, but this reflects its C++-heavy architecture rather than true memory savings. - The actual ciphertext size for TenSEAL is **334,518 bytes (~327 KB)** per encrypted vector — orders of magnitude larger than the 40-byte plaintext vector.

5.3 Approximation Error

CKKS is an approximate encryption scheme. The decrypted results are not exact but carry small numerical errors.

Operation	TenSEAL Error	OpenFHE Error
Add	6.92×10^{-9}	1.17×10^{-13}
Multiply	3.34×10^{-6}	1.74×10^{-12}
Sum	5.51×10^{-7}	1.95×10^{-14}
Average	2.94×10^{-7}	4.84×10^{-14}
Dot Product	5.11×10^{-6}	1.26×10^{-12}

Key findings: - OpenFHE achieves approximately **1,000× to 100,000× better numerical precision** than TenSEAL. - TenSEAL errors are in the range of 10^{-9} to 10^{-6}, while OpenFHE errors are 10^{-14} to 10^{-12}. - **Multiplication** introduces the largest errors in both libraries, as expected — it increases the noise in the ciphertext. - All errors are negligible for practical applications (e.g., healthcare analytics), where data precision is typically limited to a few decimal places.

5.4 Scaling with Vector Size

We measured total end-to-end time (encrypt + all operations + decrypt) at different vector sizes:

Vector Size	Plaintext (ms)	TenSEAL (ms)	OpenFHE (ms)
5	0.004	17.45	1,122.57
10	0.004	21.98	1,193.11
50	0.007	39.43	1,496.15
100	0.014	55.25	1,671.32
500	0.035	159.75	2,142.58

Key findings: - TenSEAL scales **roughly linearly** with vector size (9× increase for 100× larger input). - OpenFHE has a high **base cost** (~1,100 ms even for 5 elements) with more moderate scaling (1.9× increase for 100× larger input). - Plaintext computation remains sub-millisecond even at 500 elements. - The overhead gap narrows at larger sizes: at size 500, TenSEAL is ~13× faster than OpenFHE (vs. 64× at size 5).

6. Healthcare Use Case

As a motivating real-world scenario, we implemented a **privacy-preserving healthcare analytics** demo using TenSEAL.

Scenario: A hospital wants to outsource statistical analysis of patient blood pressure readings to a cloud provider — without exposing individual patient values.

Implementation: - Patient blood pressure readings `[120.0, 135.0, 118.0, 142.0, 128.0]` and risk weights `[0.2, 0.3, 0.15, 0.25, 0.1]` are encrypted before leaving the hospital. - The cloud computes two statistics entirely on ciphertexts: 1. **Encrypted average BP** — using sum and scalar division. 2. **Encrypted weighted risk score** — using element-wise multiplication and summation (dot product). - Only the hospital (holding the secret key) can decrypt the final results.

Results: The encrypted computations produced results matching the plaintext reference values with errors below 10^{-6}, demonstrating that HE is practical for simple statistical aggregations on sensitive data.

7. Ease of Implementation and Usability

Aspect	TenSEAL	OpenFHE
Installation	`pip install tenseal` (all platforms)	`pip install openfhe` (Linux only; requires power-of-two batch size)
API style	High-level, Pythonic (`enc + enc` , `enc.sum()`)	Low-level, explicit (`cc.EvalAdd(ct, ct)` , `cc.EvalSum(ct, n)`)
Lines of code (benchmark)	~108 lines	~161 lines
Key management	Automatic (context handles keys)	Manual (separate key generation steps for mult, sum, rotation)
Error messages	Clear Python exceptions	C++ error messages propagated through bindings
Documentation	Good Python examples, community tutorials	Academic documentation, fewer beginner resources
Cross-platform	Windows, macOS, Linux	Linux only (Python bindings)

Summary: TenSEAL is significantly easier to use for prototyping and educational purposes. OpenFHE offers more fine-grained control but at the cost of a steeper learning curve and platform limitations.

8. Practical Limitations Encountered

1. **OpenFHE platform restriction:** OpenFHE Python bindings are Linux-only. On Windows, the code must run under WSL or on a Linux server. This was a significant practical barrier.
2. **Batch size constraint:** OpenFHE requires batch sizes to be powers of two, which requires padding or rounding for arbitrary-length inputs.
3. **Memory tracking limitations:** Python's `tracemalloc` only tracks Python-side allocations. Both libraries perform most heavy computation in native C/C++ code, so reported memory figures understate true usage.
4. **Multiplicative depth budget:** CKKS ciphertexts have a limited number of sequential multiplications before the noise becomes too large. Complex computations (e.g., polynomial approximations) require careful depth planning.
5. **Serialization overhead:** A single TenSEAL ciphertext is ~327 KB for a 5-element vector. This is a major concern for network transmission in cloud computing scenarios.

9. Conclusions

This project provided a practical comparison of two homomorphic encryption libraries for basic data analytics operations. The key conclusions are:

1. **HE works, but it is expensive.** Encrypted computation is 4,400× to 113,000× slower than plaintext, depending on the library and operation. This confirms that HE is not yet suitable for latency-sensitive or high-throughput applications.
2. **TenSEAL is faster but less precise; OpenFHE is slower but more accurate.** This presents a clear speed-vs-precision tradeoff. For most practical applications (healthcare analytics, financial aggregation), TenSEAL's precision ($\sim 10^{-6}$) is more than adequate.
3. **Addition is cheap; rotation-heavy operations are expensive.** Operations like sum, average, and dot product require slot rotations that dominate the runtime in both libraries.
4. **Usability matters.** TenSEAL's Pythonic API makes it far more accessible for prototyping. OpenFHE's lower-level API provides more control but requires deeper cryptographic knowledge.

5. **Privacy-preserving analytics is feasible** for simple aggregations (averages, weighted scores) on small to medium datasets. The healthcare use case demonstrates a realistic scenario where HE adds meaningful privacy guarantees.

6. **Scaling is sublinear in both libraries** due to SIMD-style slot packing in the CKKS scheme — larger vectors don't proportionally increase computation time.

10. References

- Cheon, J. H., Kim, A., Kim, M., & Song, Y. (2017). Homomorphic encryption for arithmetic of approximate numbers. ASIACRYPT 2017.
- TenSEAL: A library for encrypted tensor operations. https://github.com/OpenMined/TenSEAL
- OpenFHE: Open-source fully homomorphic encryption library. https://github.com/openfheorg/openfhe-development
- Microsoft SEAL: https://github.com/microsoft/SEAL

This report accompanies the working implementation, benchmark code, Jupyter notebook, and generated charts submitted as part of the project deliverables.